THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Memory Management

Maria Rodriguez Read

Semester 1, 2025

© University of Melbourne

Why Memory Management?

- Support multiprogramming
  - To improve CPU utilisation, multiple processes need to be loaded into memory to ensure a reasonable supply of ready processes to consume available CPU time
- Security – isolation between processes and OS memory
- Enable a computer system to run more processes with larger memory requirements, e.g.:
  - A single process can use more memory than what is physically available
  - The sum of the memory requirements of all processes can be larger than what is physically available

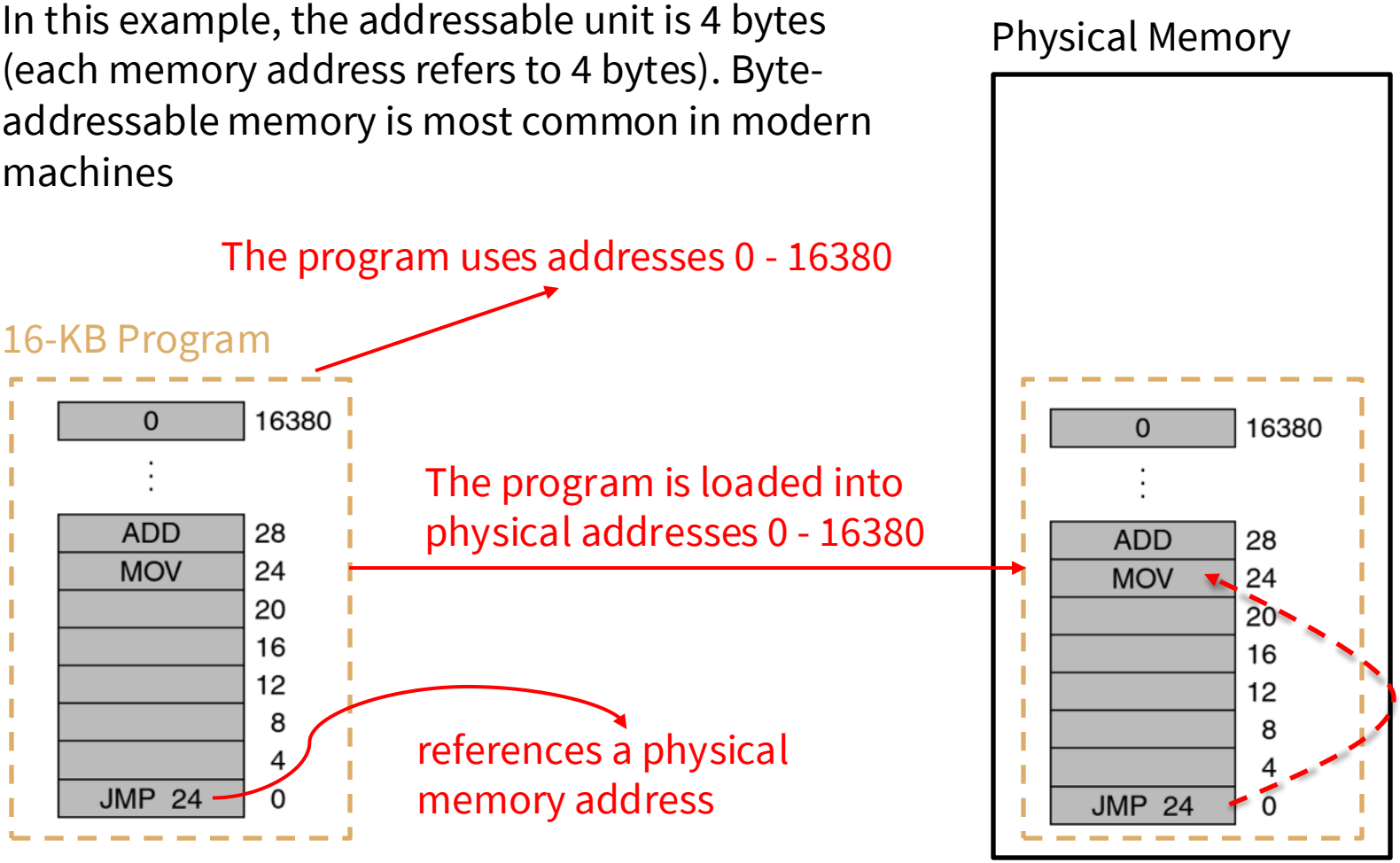
© University of Melbourne

Early Systems – No Memory Abstraction

- Programs referenced physical memory directly
- For example:
  - MOV REGISTER, 1000
  - Copy the memory contents at physical address 1000 to REGISTER
- One way of achieving multiprogramming under this model?
  - Have the running process in memory and all other processes on disk. On a context switch, the running process is swapped to disk, and the newly scheduled process is swapped into memory

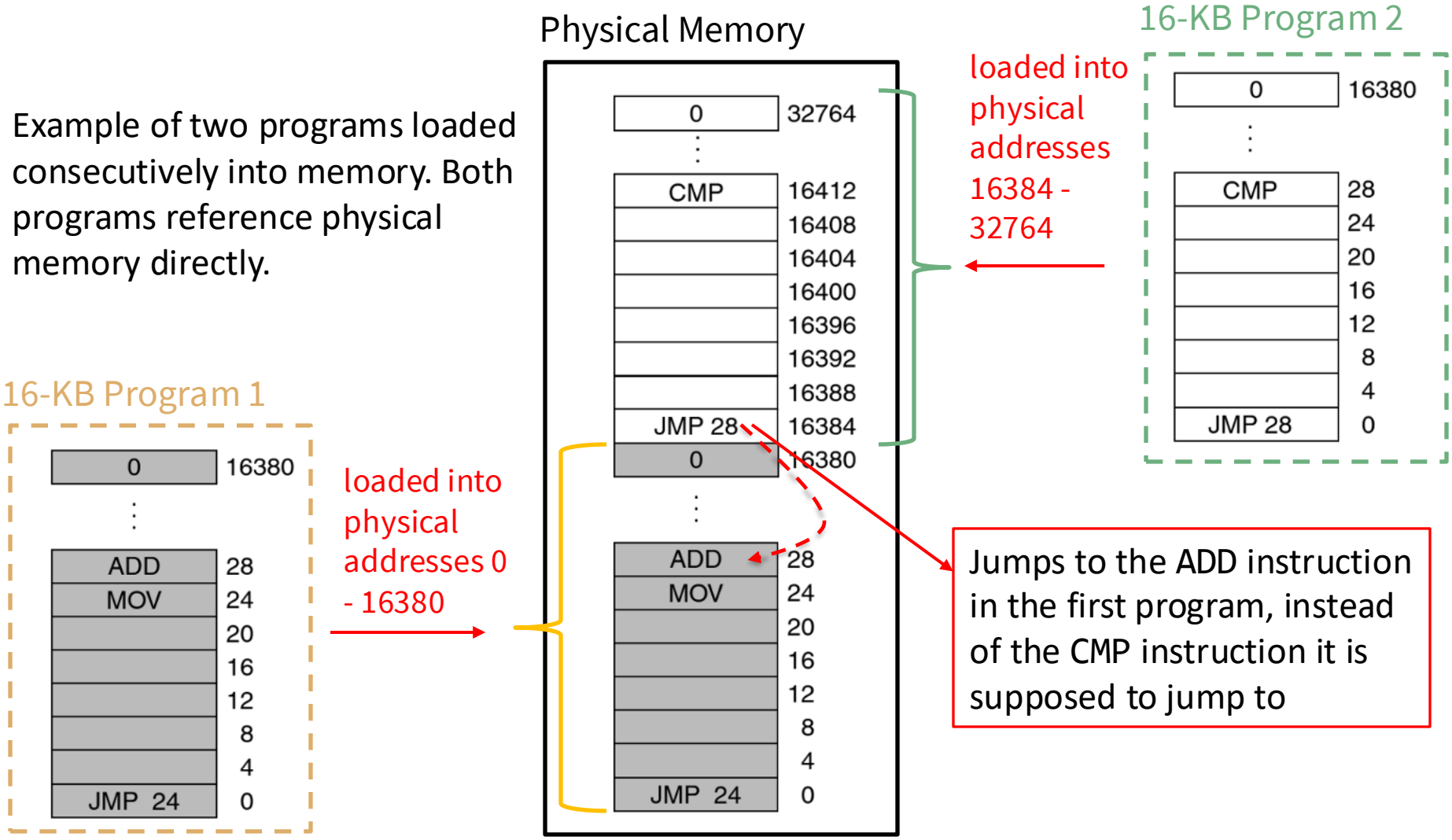
Single Processes in Memory with No Abstraction

In this example, the addressable unit is 4 bytes (each memory address refers to 4 bytes). Byte-addressable memory is most common in modern machines



Multiple Processes in Memory with No Abstraction

Example of two programs loaded consecutively into memory. Both programs reference physical memory directly.



Jumps to the ADD instruction in the first program, instead of the CMP instruction it is supposed to jump to

Memory Abstraction

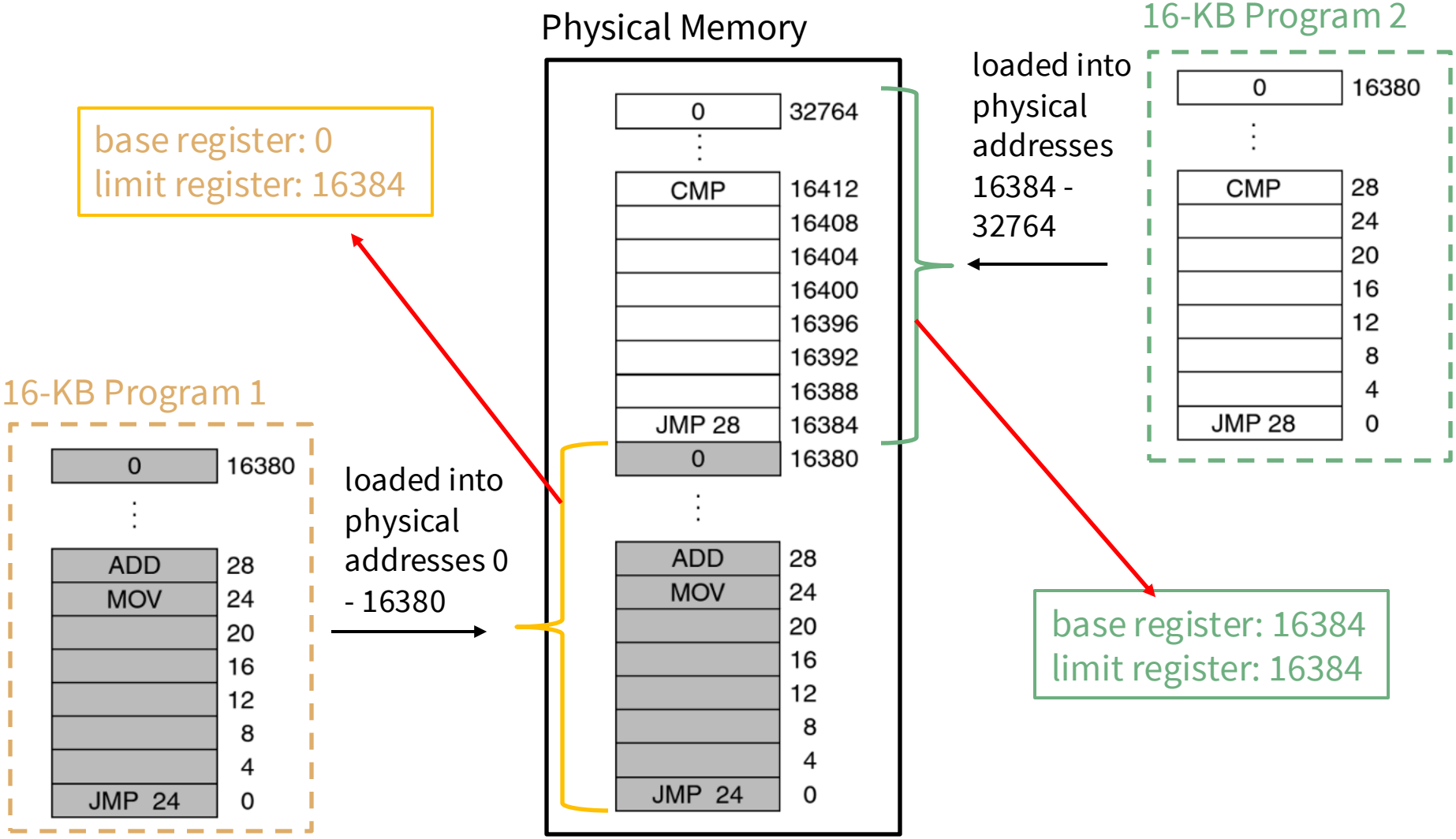
- Provide a logical address space that is bound to a separate physical address space
  - Program instructions refer to *logical addresses*
  - The OS manages physical memory
  - The hardware translates logical addresses to physical addresses at runtime
- We will look into two approaches to memory abstraction:
  - Dynamic relocation with base and limit registers (base and limit registers for short)
  - Paged virtual memory

Base and Limit Registers

- A program is loaded into consecutive memory addresses wherever there is room
- Two registers in the CPU are used to delimit the physical memory location of the process:
  - **Base register**: contains the physical address where the program begins in physical memory
  - **Limit register**: contains the length of the program
- The values of the registers are determined by the OS and maintained in the PCB. They are loaded into the CPU when the process is allocated to the CPU

© University of Melbourne

Base and Limit Registers



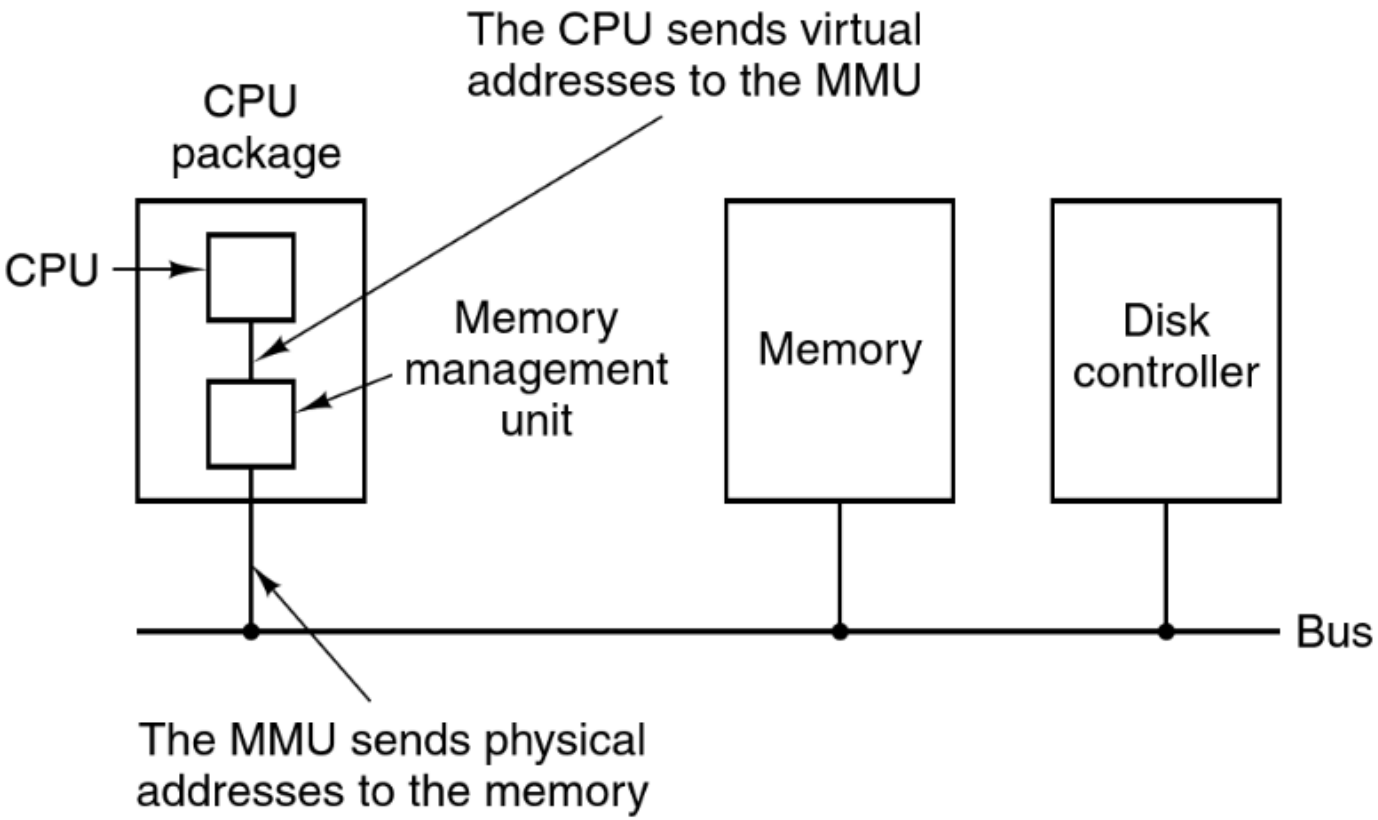
Base and Limit Registers – Translating Logical to Physical Addresses

- Every time a process references memory, the CPU hardware:
  1. Checks that the **logical address is *less than* the value of the limit register**. If it is not, the logical address is not within the bounds of the process, and the CPU raises an exception (interrupt)
  2. If the logical address is within the bounds, the hardware **adds the base value to the logical address** before sending the (physical) address out on the memory bus

```
physical address = logical address + base
```

- The CPU hardware that performs address translation is the memory management unit (MMU)

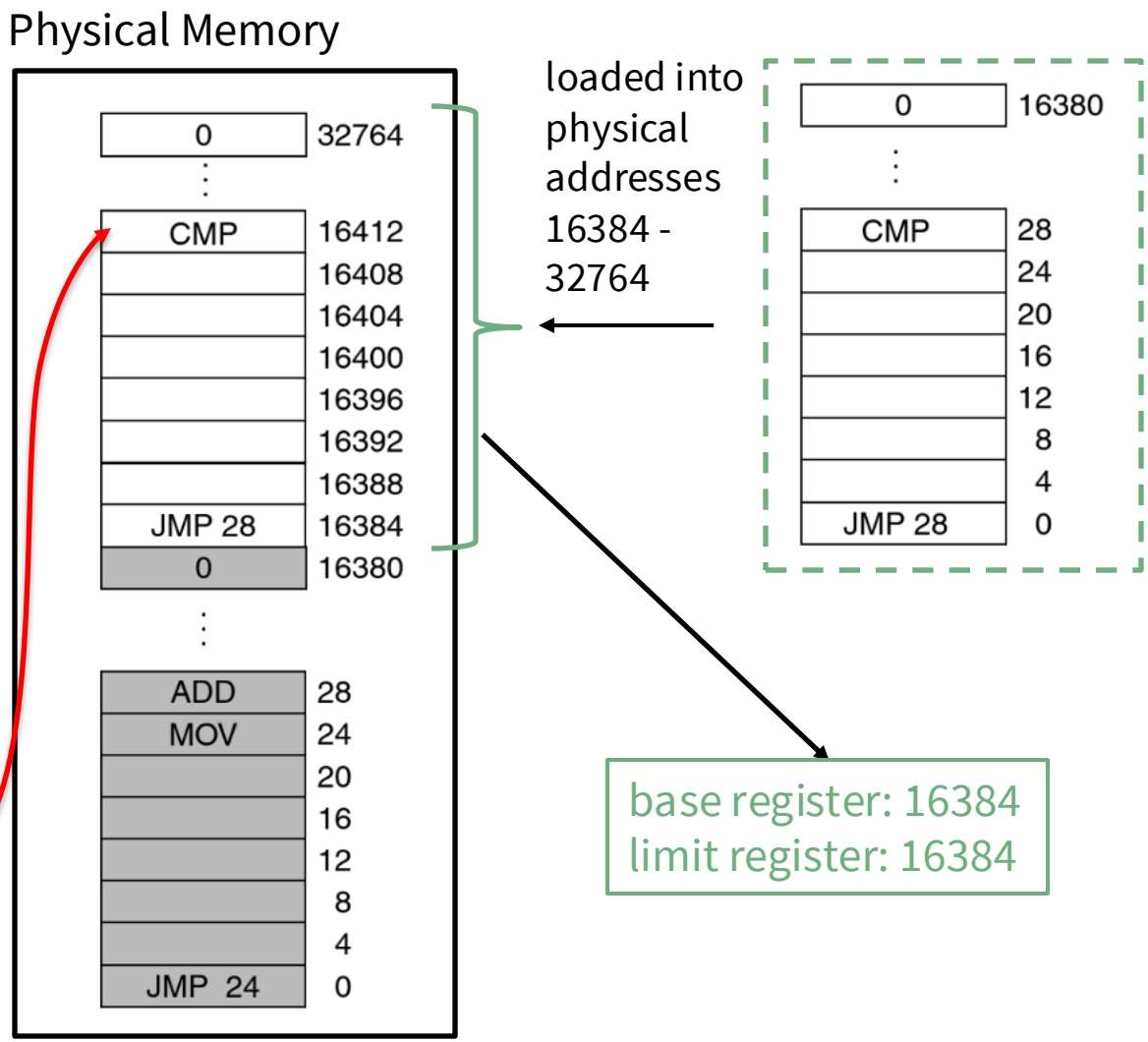
Memory Management Unit



Base and Limit Registers – Example

When JMP 28 executes, the MMU:

1. Checks the bounds of the address: is 28 < 16384? Yes
2. Generates the physical address as 28 + 16384 = 16412



Memory Management with Base and Limit Registers

- Requires contiguous memory allocation
- OS needs to manage physical memory by:
  - Keeping track of memory usage
  - Allocating free (contiguous) memory to processes
  - Deallocating memory when a process is swapped to disk or terminates

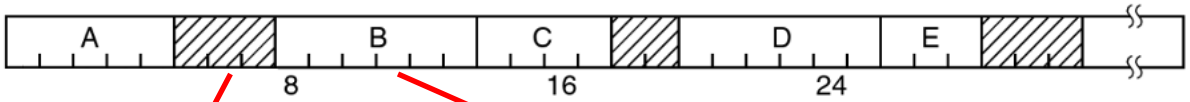
Keeping Track of Memory Usage

- One approach is to use a linked list
- Each entry in the list specifies:
  - A hole (H) or process (P)
  - The address at which it starts
  - The length of the hole or the allocation
  - A pointer to the next entry

© University of Melbourne

Linked List Example

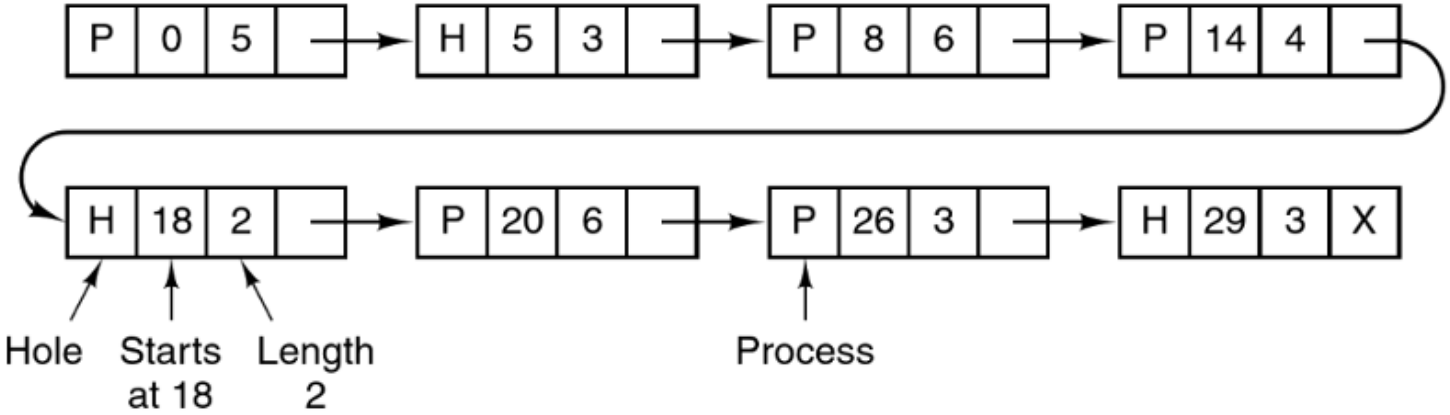
Memory state:



free memory in addresses 5, 6, 7 (hole)

process B allocated to physical addresses 8 - 13

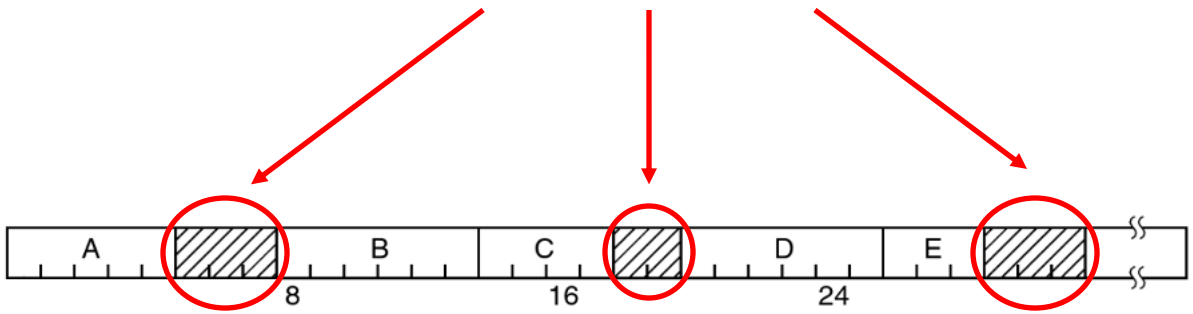
Corresponding linked list (sorted by address):



Allocating Contiguous Memory

- Choosing a free block of memory to allocate to a process
- Once a free block is chosen, it is divided into 2 pieces: one for the process and one for the unused memory, except in the statistically unlikely case of an exact fit.

Which free block should the process be allocated to?



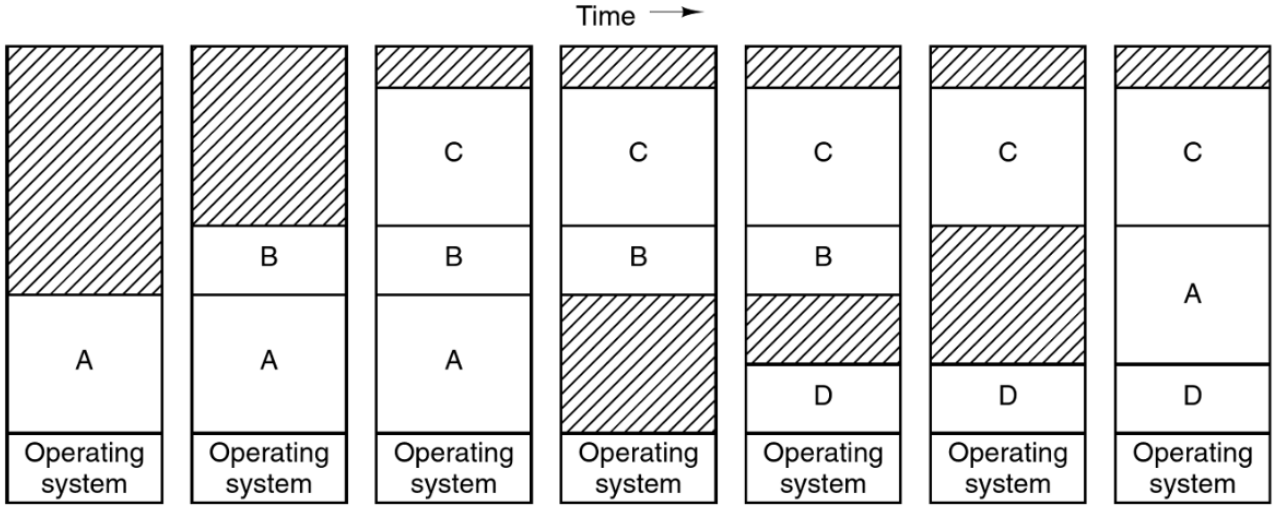
Allocating Contiguous Memory

- First-fit
  - Allocate the process to the first free block that fits the process
  - Fast, it searches as little as possible
- Best-fit
  - Search the entire list and allocate the process to the smallest free block that fits the process
  - Slower than first-fit
  - Tends to create small, useless free blocks because it produces the smallest leftover memory hole
- Worst-fit
  - Allocate the process to the largest available free block
  - Produces the largest leftover memory hole

© University of Melbourne

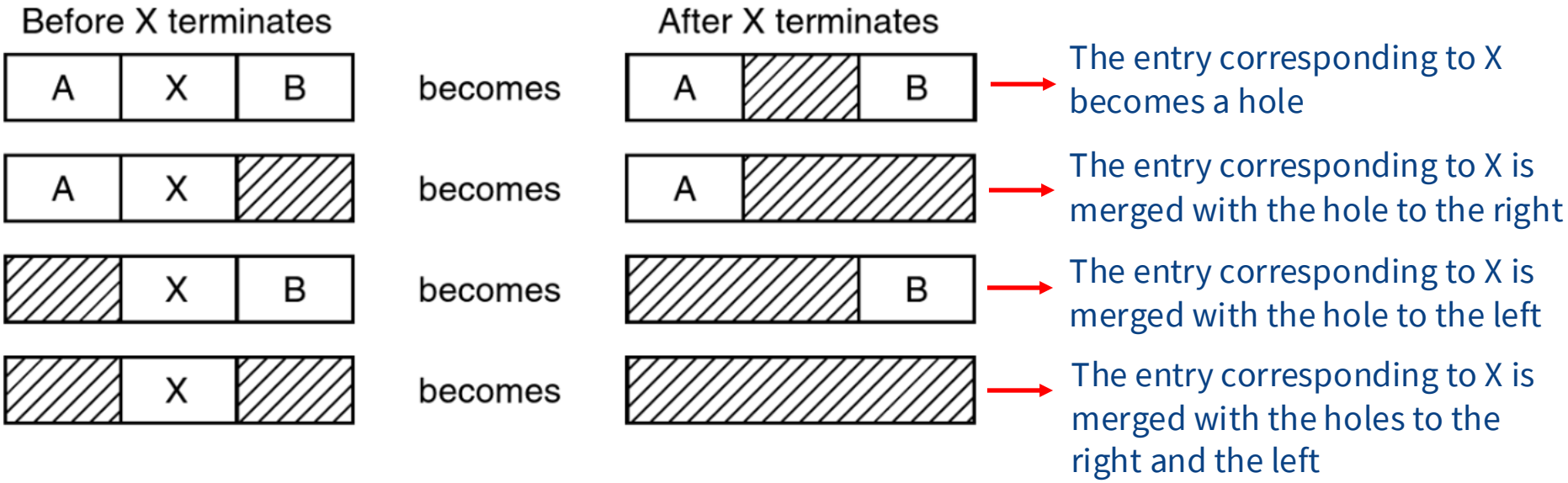
Deallocating Contiguous Memory

- Required when a process terminates or is swapped out of memory
- Swapping
  - When not running, a process can be swapped out of memory to a backing store (e.g., disk) and then brought back into memory when it resumes execution
  - Makes it possible for the total physical address space of all processes to exceed the real physical memory of the system



Deallocating Contiguous Memory

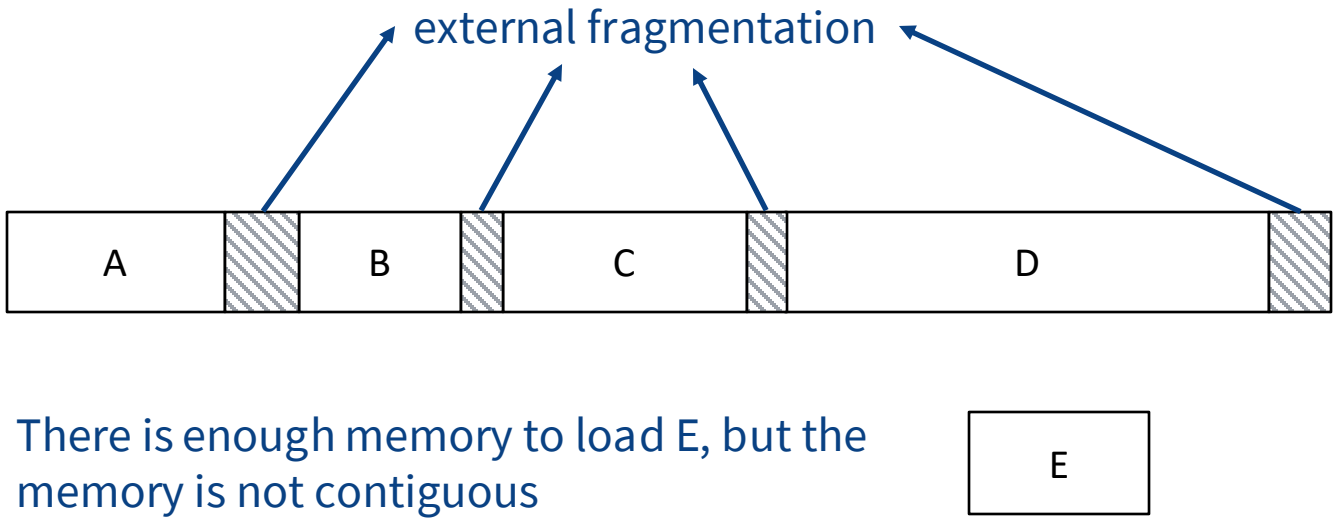
- The data structure representing the state of physical memory must be updated accordingly



TB 3-7

External Fragmentation

- As processes are loaded and removed from memory, free memory might be broken into very small pieces in such a way that there is enough total memory space to satisfy a request, but the available spaces are not contiguous



Limitations of Base and Limit Registers

- An entire process must be loaded into contiguous memory addresses
- A process cannot be larger than physical memory
- When more space is needed, an entire process needs to be swapped to disk
- External fragmentation
- If the memory usage of a process grows, the OS may need to move it

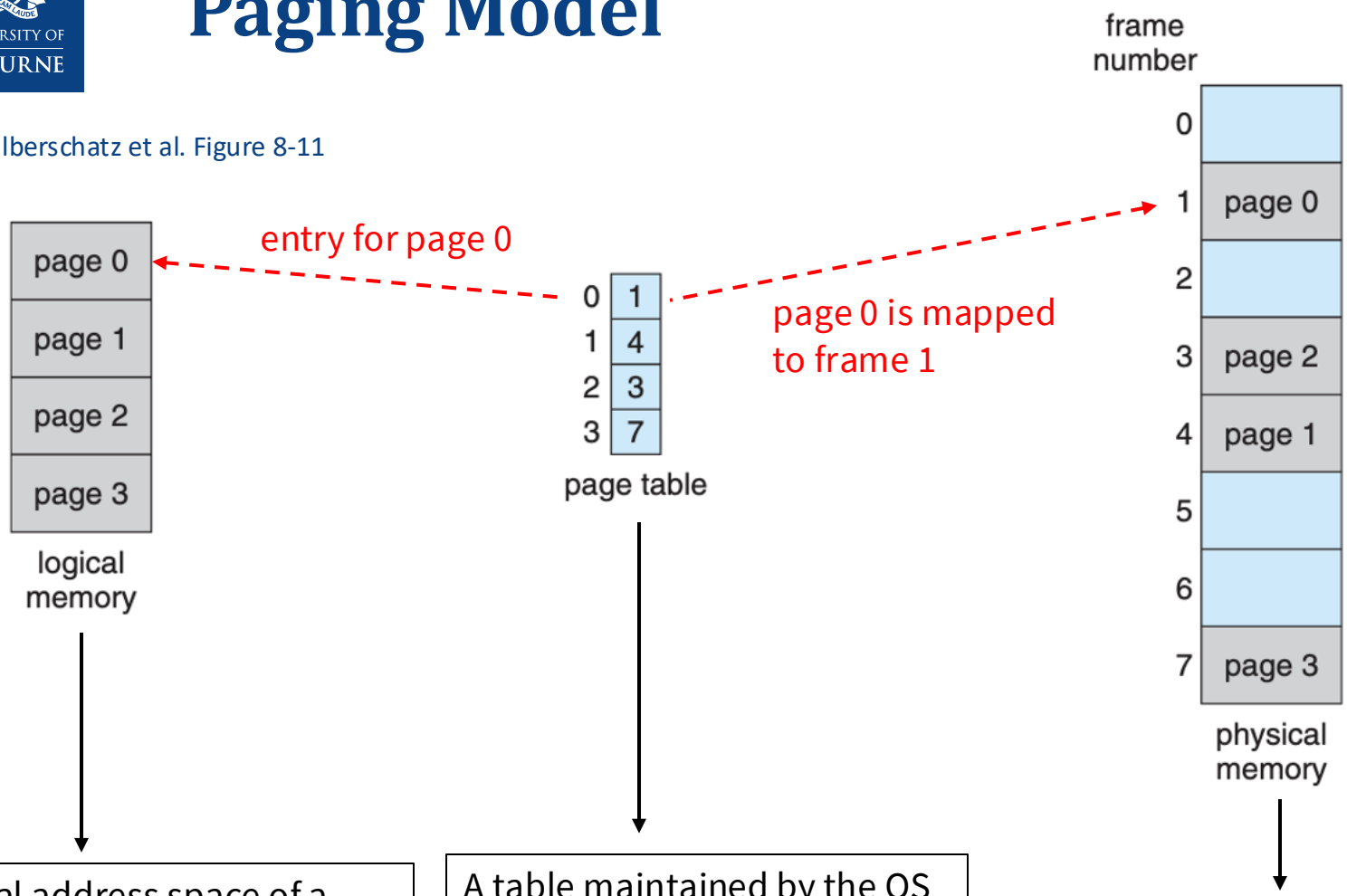
Paged Virtual Memory

- Break up the address space of a process into fixed-sized pieces called **pages**
- Break up physical memory into fixed-sized slots called page **frames**
- Pages are mapped to frames
  - Pages and frames are generally the same size. Each frame contains a single page
  - Pages don't need to be stored in contiguous frames
- Not all pages have to be in physical memory at the same time to run a process
  - During the lifetime of a process, pages start out on disk and may move between disk and memory any number of times
  - When a page is moved to disk and back again, it may be put in a different page frame than before

© University of Melbourne

Paging Model

Silberschatz et al. Figure 8-11



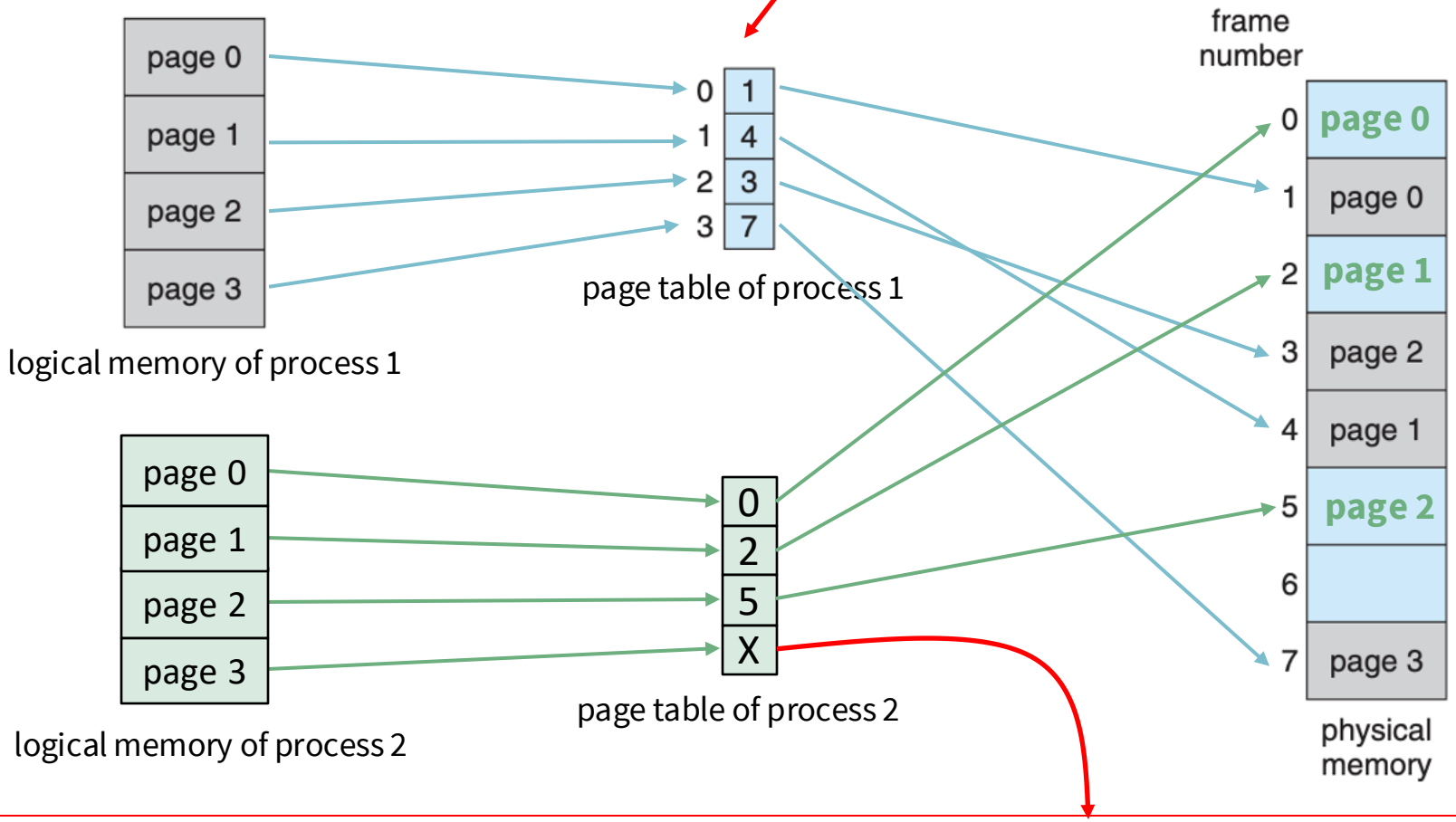
Logical address space of a process broken down into 4 pages, pages 0 to 3

A table maintained by the OS (per process) that indicates where each page is placed in physical memory

Physical memory is broken down into 8 frames, frames 0 to 7

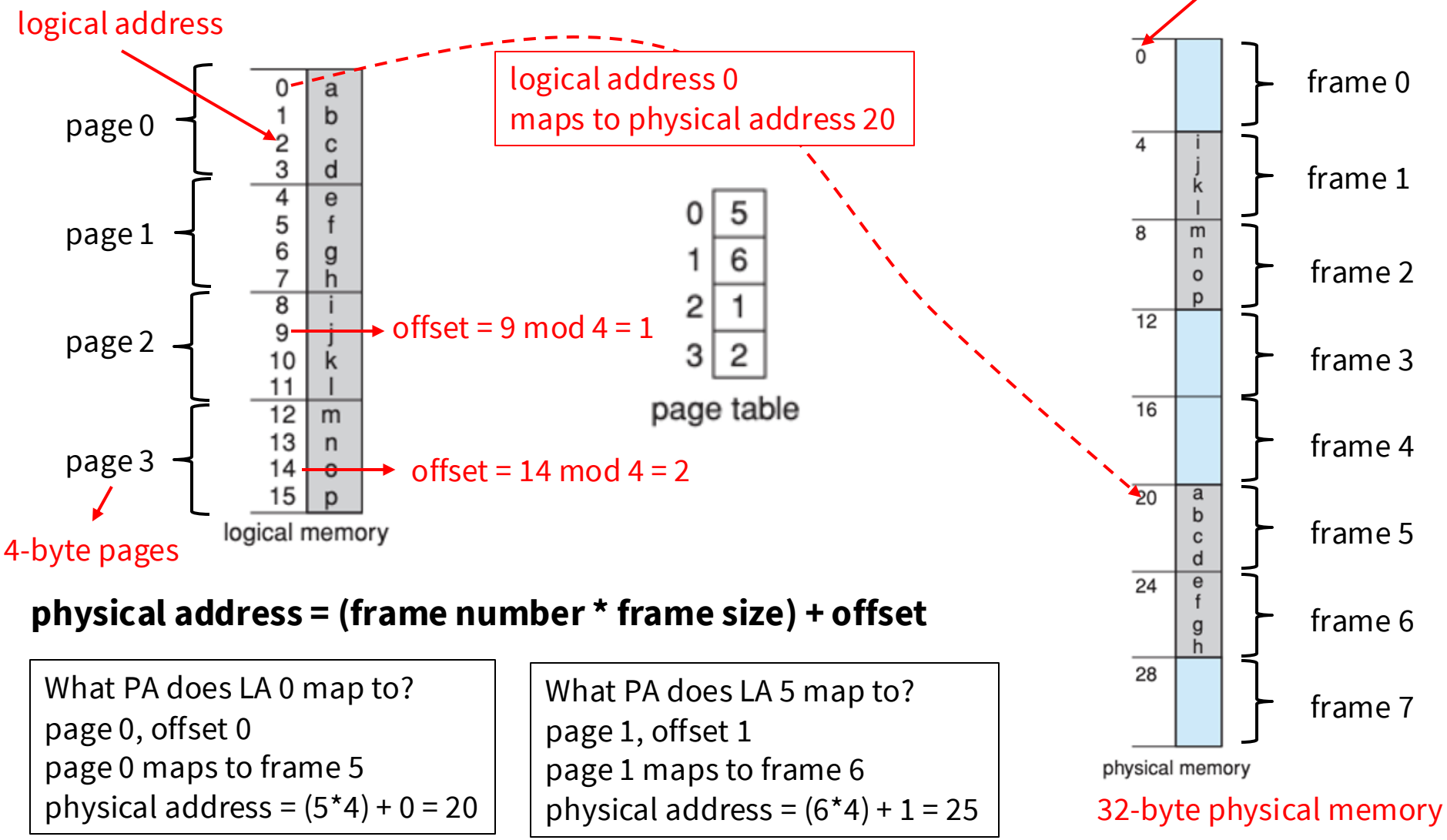
Paging Model

One page table for each process, stored in memory. Page Table Base Register (PTBR) points to the start of the page table (maintained in the PCB)



In this case, page 3 is not in memory (indicated by X). Page 3 is on disk and will be brought into memory when one of its addresses is referenced by a running instruction

Paging Example



physical address = (frame number * frame size) + offset

What PA does LA 0 map to?
page 0, offset 0
page 0 maps to frame 5
physical address = (5*4) + 0 = 20

What PA does LA 5 map to?
page 1, offset 1
page 1 maps to frame 6
physical address = (6*4) + 1 = 25

Structure of a Logical Address

Logical address:

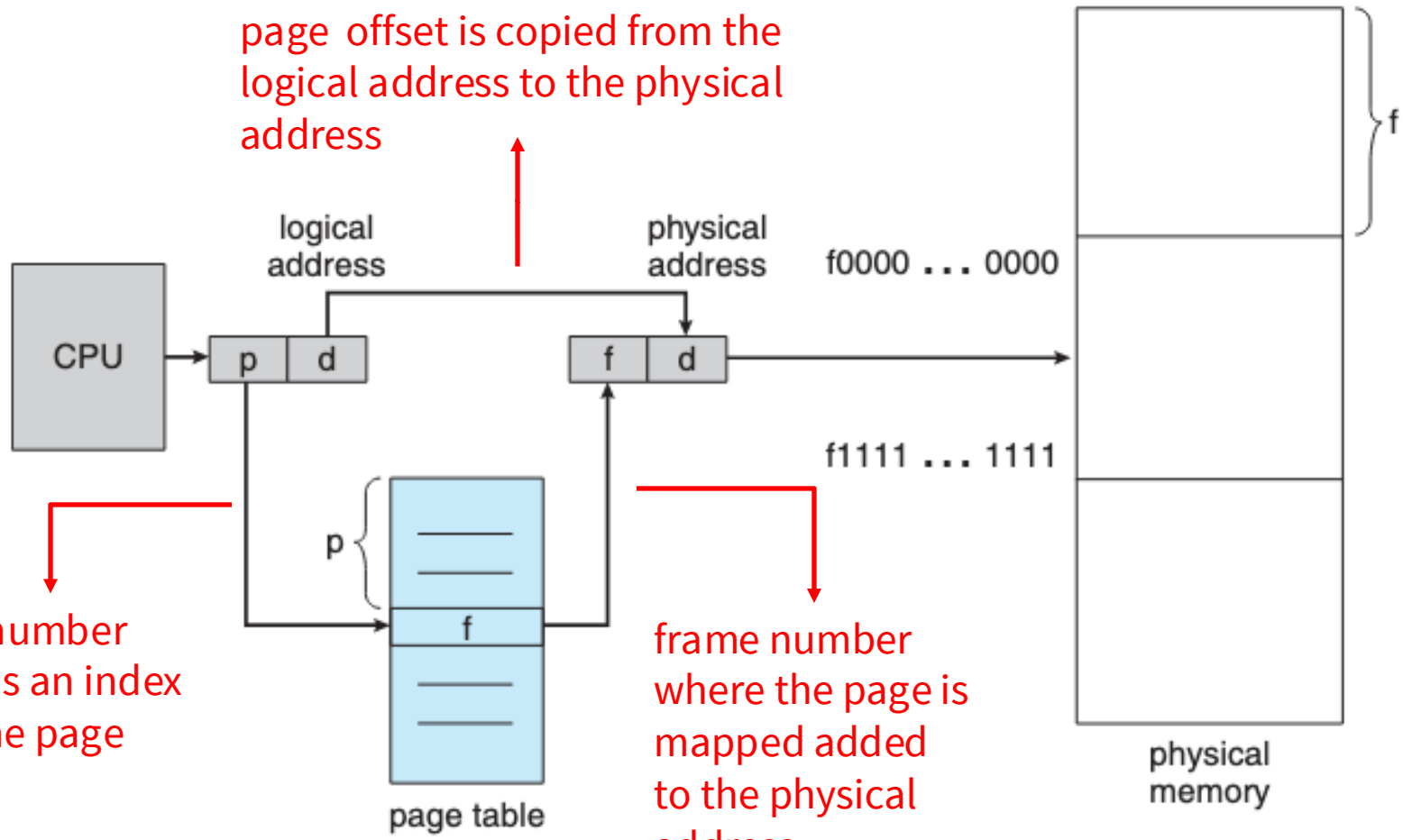
The first *m-n* bits represent the page number

The last *n* bits represent the page offset

- *m* bits to represent the logical address (the size of the logical address space is $2^{m)}$
  - In the previous example, the logical address space is $2^4 = 16$. A logical address would be 4 bits long
- *n* bits to represent the page offset (the page size is 2^n)
  - In the previous example, the page size is $2^2 = 4$. 2 bits in the logical address are used for the page offset (which can be 0, 1, 2, 3)
- *m-n* bits to represent the page number (process can have $2^m / 2^n = 2^{m-n}$ pages)
  - In the previous example, a process can have $2^4 / 2^2 = 2^2 = 4$ pages. 2 bits are used for the page number

MMU in a Paged System

page offset is copied from the logical address to the physical address

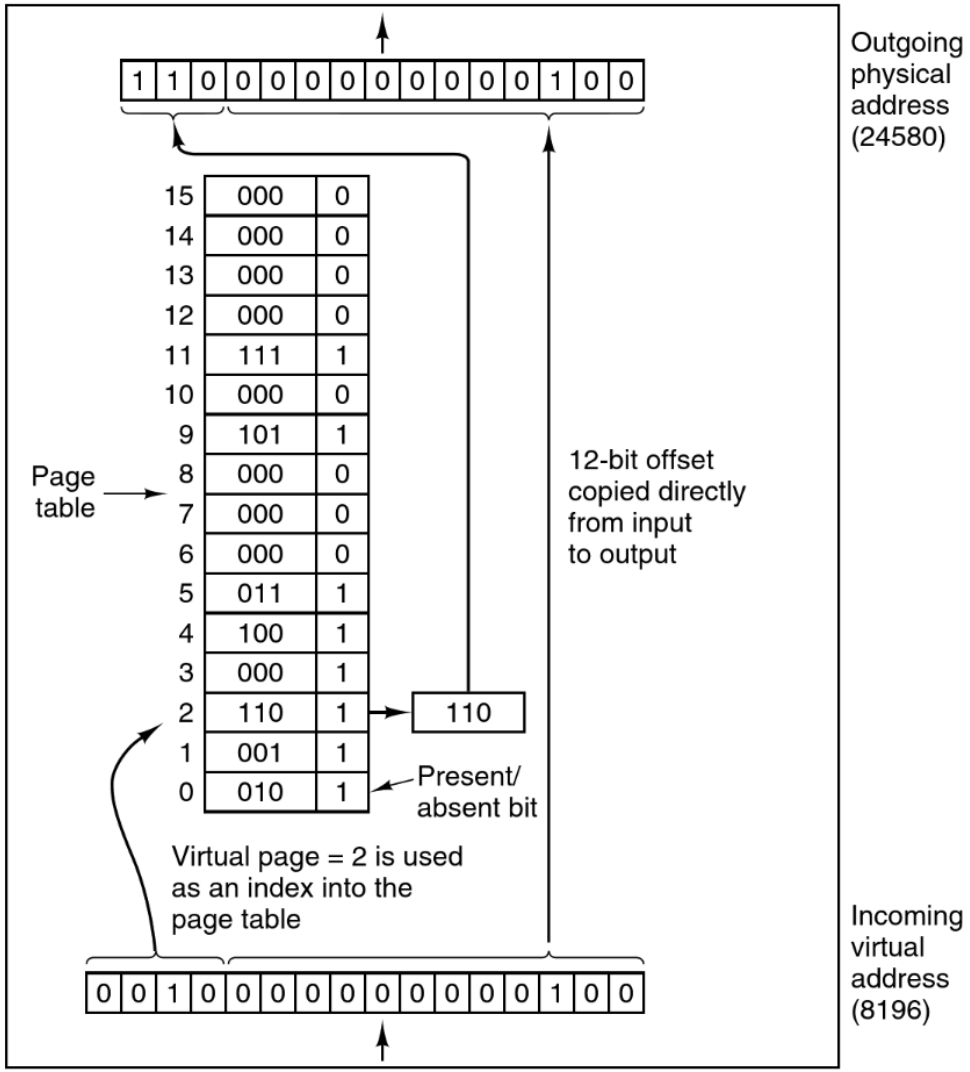


page number used as an index into the page table

frame number where the page is mapped added to the physical address

Silberschatz et al. Figure 8-10

Address Translation Example



TB 3-10

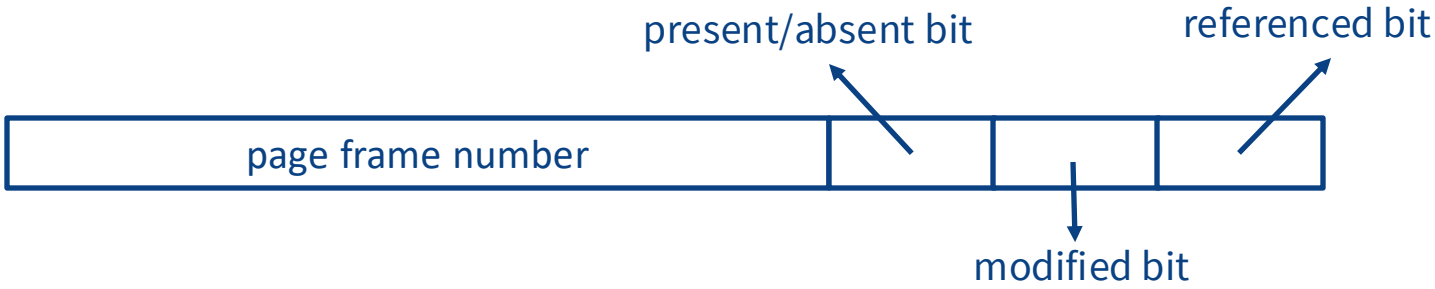
Internal Fragmentation

- When a process is divided into fixed-sized pages, there may be some leftover space on the last page (internal fragmentation)
- Example:
  - Page size = 4 bytes
  - Process size = 13 bytes
  - Process needs 4 pages
  - 4 pages = 16 bytes
  - Internal fragmentation: 16 bytes – 13 bytes = 3 bytes (wasted)
- Effect of page size?
  - Larger page sizes
    - More internal fragmentation
    - Smaller page tables (unrelated to fragmentation but a trade-off to consider)
  - Smaller page sizes
    - Less internal fragmentation
    - Larger page tables

© University of Melbourne

Structure of a Page Table Entry

- Aside from the frame number, a page table entry contains other useful information about the page. In this subject, we are interested in:
  - Present/absent bit
    - Indicates whether the page is in physical memory (1) or on disk (0)
    - If 0, the MMU will NOT translate the address and will trigger a **page fault** (interrupt)
  - Referenced bit
    - When a page is referenced (accessed), the MMU automatically sets the Referenced bit to 1
  - Modified bit
    - When a page is written to, the MMU automatically sets the Modified bit to 1



Page Fault

- Some pages of a process may not be mapped to frames, i.e., are not in physical memory and are in a backing store (disk)
- If a process references an address of a page not currently mapped to a frame, the MMU raises a page fault through an interrupt
- On a page fault, the OS
  - If there are no free frames, picks a page frame to evict and writes its contents to disk (only if the modified bit is set, if not, just discard the page)
  - Fetches the page that was referenced (from disk) and loads it into the page frame that was just freed
  - Updates the page table
  - Restarts the process at the instruction that caused the page fault

© University of Melbourne

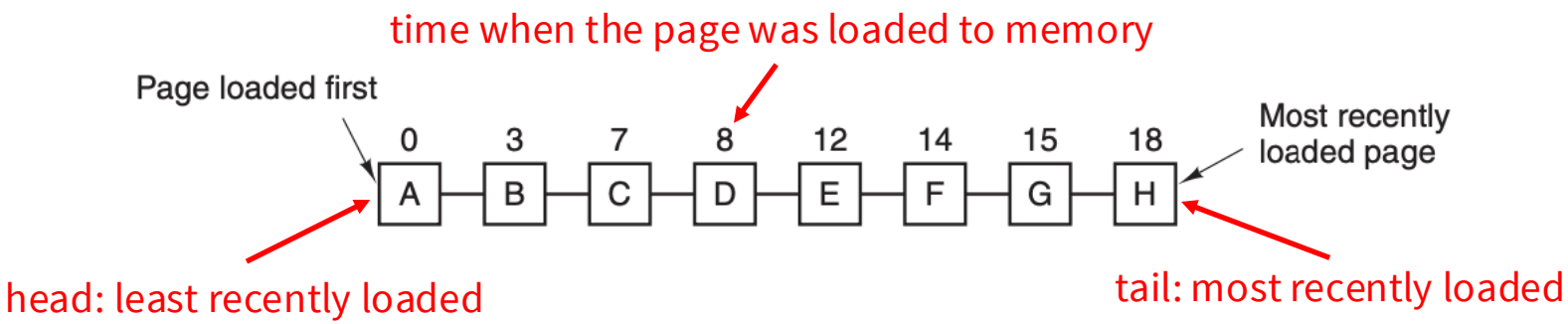
Page Replacement Algorithms

- How to pick a page frame to evict when a page fault occurs?
- Page replacement algorithms:
  - Optimal
    - Leads to the **fewest number of page faults** overall
    - Evict the page that will be accessed furthest in the future
    - Useful as a comparison point but not practical
  - First-in, First-out (FIFO)
  - Second-chance
  - Least Recently Used (LRU)

© University of Melbourne

First-in, First-out (FIFO)

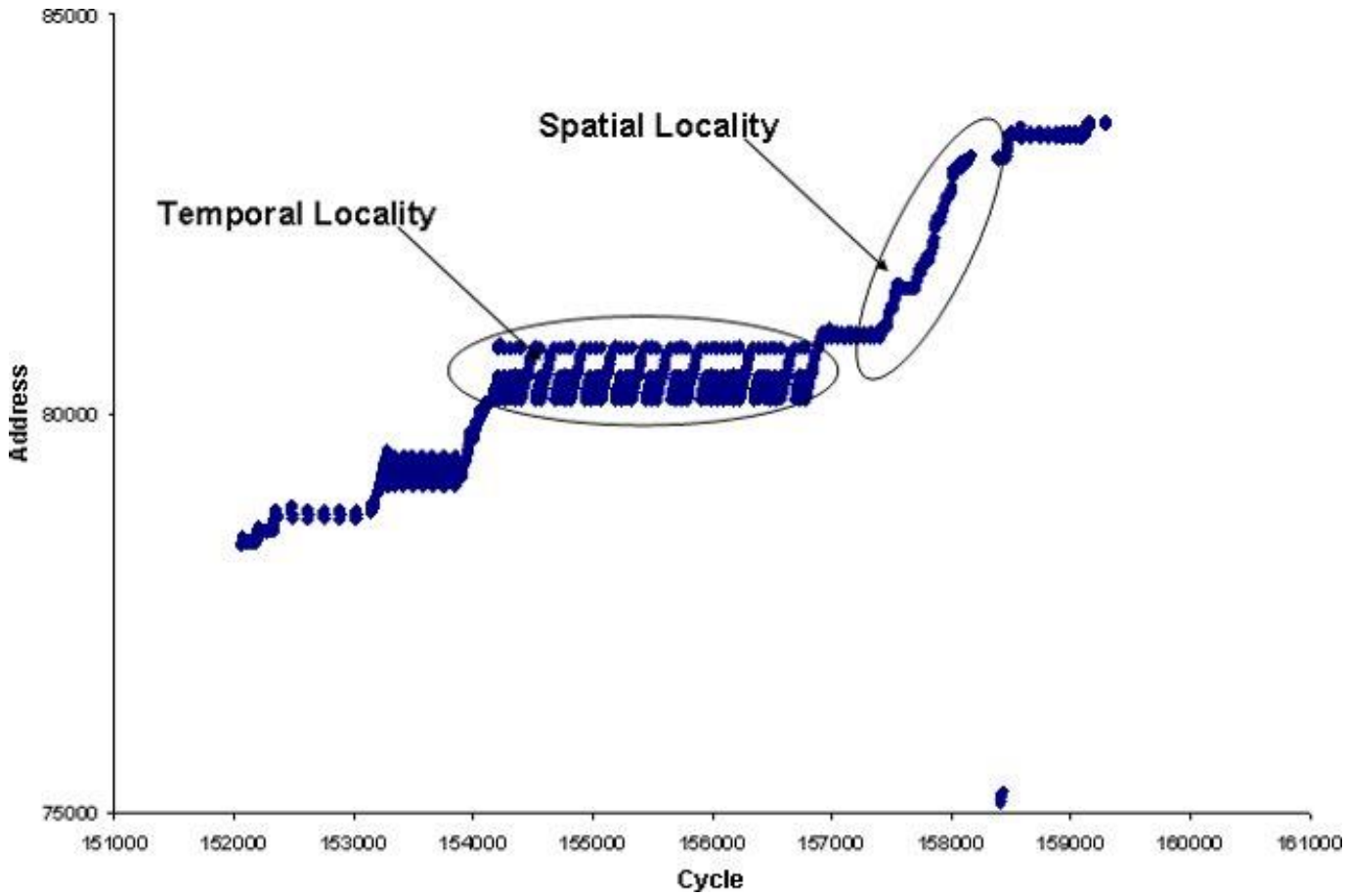
- Evict the page that has been in memory the longest
- Maintain a FIFO queue of pages based on load time
  - Page loaded the longest ago at the head of the queue
  - Most recently loaded page at the tail of the queue
  - Evict the head, put newly mapped page at tail
- Simple to implement, but may evict heavily used pages



Principle of Locality

- Most programs exhibit temporal and spatial locality
  - **Temporal locality**: if a process accesses a particular memory address, it will likely reference the same address in the near future
  - **Spatial locality:** if a process accesses a particular memory address, it will likely reference nearby addresses in the near future
- (some) page replacement algorithms that consider this principle:
  - Second-chance
  - LRU

Principle of Locality - Example

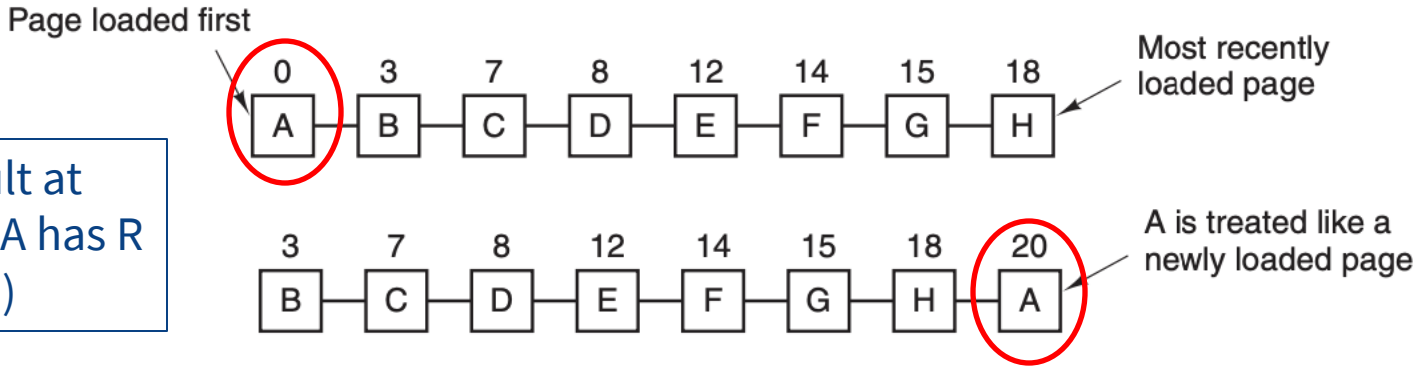


© University of Melbourne

Second-chance Algorithm

- Simple modification to FIFO to avoid evicting a recently used page
- Inspect the *Referenced (R) bit* of the oldest page
  - If 0, the page is old and not recently used, evict
  - It 1, the page is old but recently used
    - Clear the R bit
    - Move the page to the tail of the queue (give it a second chance)
    - Update the load time of the page as if it had just arrived in memory

Page fault at time 20. A has R bit set (1)



Least Recently Used (LRU)

- Pages that have been accessed in the near past are likely to be accessed again in the near future
- Evict the least-recently-used page (i.e., the page that has not been used for the longest time)
- Possible implementation:
  - Maintain a list of all pages with the most recently used page at the head and the least recently used at the tail. The page at the tail of the list is chosen for eviction
  - On every memory reference, find the page that was referenced in the list and move it to the head
- Theoretically realisable, but has high performance overhead

© University of Melbourne

LRU Approximation - Aging

- Maintain a *bit counter* for each page
- At each clock interrupt (clock tick)
  - Shift all counters by one bit to the right
  - Append the *Referenced bit* to the leftmost bit (append 1 if the page was referenced since the last clock tick, or 0 if it wasn't)
- At page fault, evict the page with the lowest counter

© University of Melbourne

Aging Example

R bits for pages 0-5, clock tick 0

1	0	1	0	1	1

R bit for page 0 1 2 3 4 5

Page	
0	10000000
1	00000000
2	10000000
3	00000000
4	10000000
5	10000000

8-bit counter for each page

clock tick 0

Aging Example

R bits for pages 0-5, clock tick 0

1	0	1	0	1	1

R bit for page 0 1 2 3 4 5

R bits for pages 0-5, clock tick 1

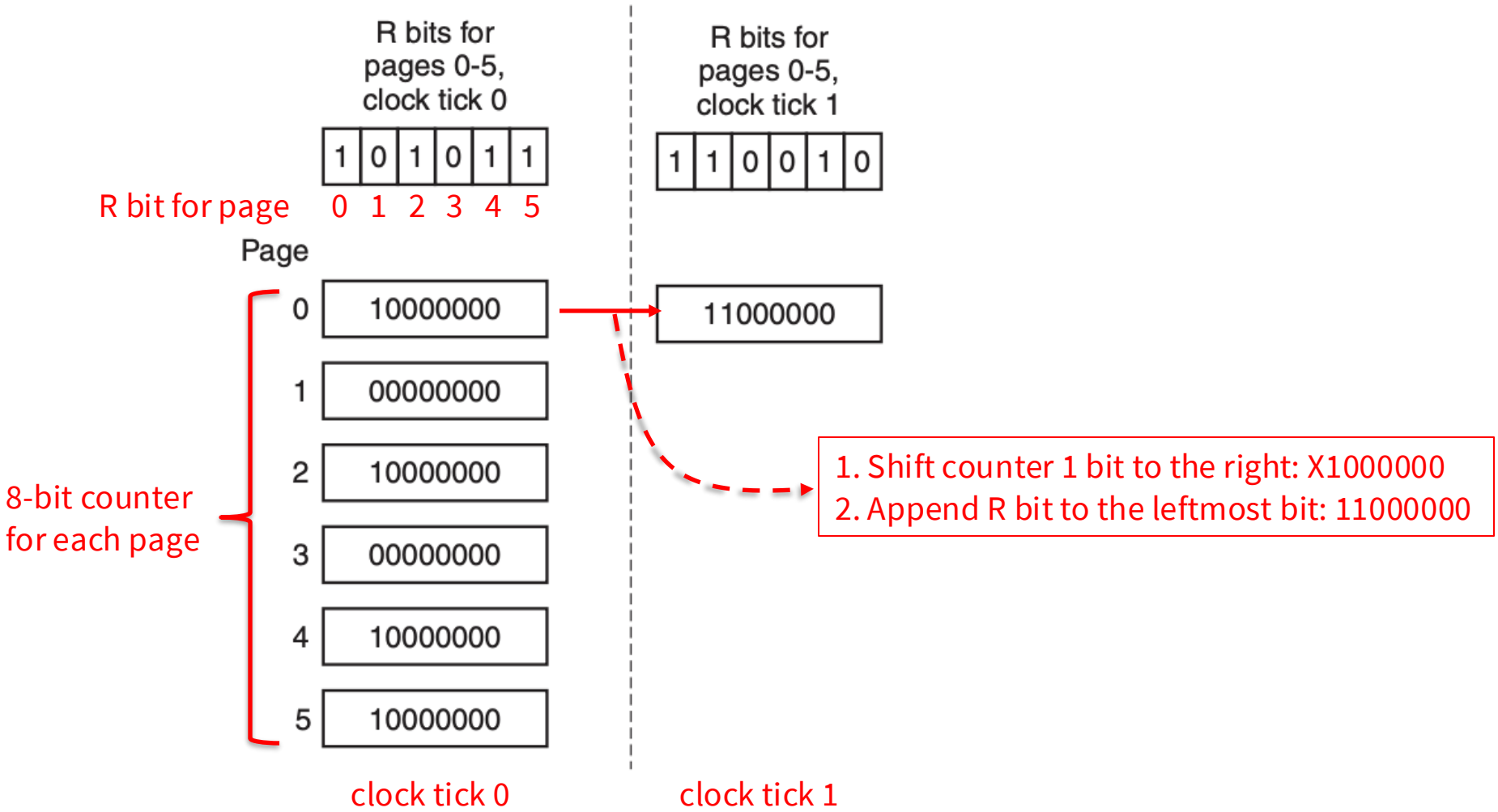
1	1	0	0	1	0

8-bit counter for each page

Page	
0	10000000
1	00000000
2	10000000
3	00000000
4	10000000
5	10000000

clock tick 0 clock tick 1

Aging Example



Aging Example

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1
	1 0 1 0 1 1	1 1 0 0 1 0
R bit for page	0 1 2 3 4 5	

Page	clock tick 0	clock tick 1
0	10000000	11000000
1	00000000	10000000
2	10000000	01000000
3	00000000	00000000
4	10000000	11000000
5	10000000	01000000

8-bit counter for each page

Aging Example

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1
R bit for page	0 1 2 3 4 5		
Page			
0	10000000	11000000	11100000
1	00000000	10000000	11000000
2	10000000	01000000	00100000
3	00000000	00000000	10000000
4	10000000	11000000	01100000
5	10000000	01000000	10100000
	clock tick 0	clock tick 1	clock tick 2

8-bit counter for each page

Page 1 not referenced in 2 clock ticks, has 2 leading 0s

Page 4 not referenced in 1 clock tick, has 1 leading 0

Full example in TB Figure 3-17

Limitations of Aging

- Aging may evict a page that was **not the least recently used**
- By recording only 1 bit per time interval (clock tick), the algorithm cannot distinguish between references that occurred early in the interval and those that occurred later
- Counters have a finite number of bits, which limits the algorithm's past horizon.
  - E.g., If 2 pages have a counter value of 0, all we can do is pick one of them at random. In reality, it may well be that one of the pages was last referenced 9 ticks ago, and the other was last referenced 1000 ticks ago

© University of Melbourne

Translation Lookaside Buffer (TLB)

- Without optimisation, in a paged system, each memory reference requires 2 memory accesses: one to access the page table and one to access the data
- The TLB is a fixed-capacity hardware cache in the MMU
- Stores copies of recently accessed **page table entries**
- Implemented using associative circuitry (all entries are looked up in parallel)
- Each entry in the TLB contains information about one page

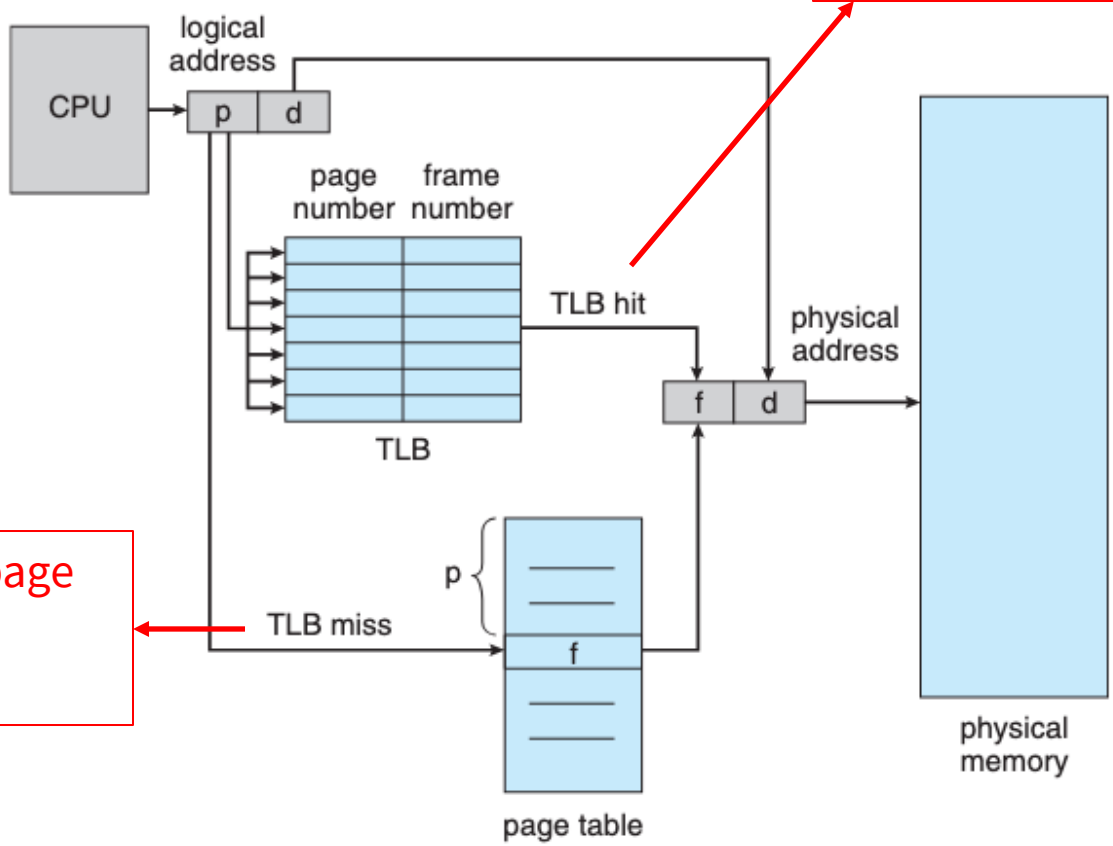
Sample TLB entry:

TLB valid bit	virtual page number	page frame number	… other bits

© University of Melbourne

TLB

an entry for the page number was found in the TLB



an entry for the page number was not found in the TLB

TLB Discussion

- On a TLB miss
  - An entry from the TLB is evicted (if the TLB is full) and replaced with an entry for the page that was just looked up
- TLB valid bit
  - Indicates whether the cached entry is valid or not
- If a page of the currently running process is swapped out
  - The corresponding entry in the TLB becomes invalid. The page is not mapped to the frame anymore
  - The entry in the TLB is invalidated
- On a context switch
  - The TLB contains mappings that are only valid for the currently running process (the same virtual page maps to different physical frames for different processes)
  - One approach is to flush the TLB on a context switch (invalidate all entries)
  - The newly running process will encounter many TLB misses while the TLB is re-populated with valid entries
- Effect of page size on TLB hits/misses?

© University of Melbourne

References

– Abraham Silberschatz, Greg Gagne, and Peter Baer Galvin, Operating System Concepts, Ninth Edition.